

Universidade de Vigo

ESCOLA SUPERIOR DE ENXEÑARÍA INFORMÁTICA

Memoria do Traballo de Fin de Grao que presenta

D. Saúl Candán Vidal

para a obtención do Título de Graduado en Enxeñaría Informática

Implementación de "counting sort" en una GPU



20 de Agosto, 2023

Traballo de Fin de Grao N°: EI 22/23-105

Titor/a: Arno Formella

Área de coñecemento: Linguaxes e Sistemas Informáticos

Departamento: Informática

Dedicatoria

A todos esos amigos y familiares que me hacen feliz gracias a las risas y experiencias compartidas.

Agradecimientos

A Paco y Geli, por el infinito esfuerzo realizado para darme el apoyo económico que hizo posible mis estudios.

Índice de contenidos

1	Introducción.....	1
2	Objetivos.....	2
3	Solución propuesta.....	3
3.1	Funcionamiento del algoritmo Counting Sort.....	3
3.1.1	Fase de conteo.....	3
3.1.2	Fase de generación de el array ordenado.....	4
	Método1:.....	4
	Método2:.....	4
3.2	Diseño paralelo propuesto del algoritmo Counting Sort.....	6
3.2.1	Fase de conteo.....	6
3.2.2	Fase de generar array ordenado.....	8
	Método1:.....	8
	Método2:.....	9
4	Metodología Usada.....	13
5	Planificación y seguimiento.....	14
6	Arquitectura.....	16
7	Tecnologías e integración de productos de terceros.....	21
8	Especificación y análisis de requisitos.....	22
9	Manual de usuario.....	24
9.1	Requisitos Mínimos.....	24
9.2	Guía de Instalación.....	25
9.2.1	Instalación de Microsoft Visual Studio.....	25
9.2.2	Instalación de CUDA.....	25
9.3	Manual de Utilización.....	26
10	Pruebas.....	28
11	Conclusiones y aportaciones.....	34
11.1	Principales aportaciones.....	34
11.2	Conclusiones.....	34
11.3	Vías de trabajo futuro.....	34
12	Referencias.....	35

Índice de ilustraciones

Ilustración 1. Metodología TTD.....	13
Ilustración 2. Diagrama de Gantt - Planificación inicial.....	14
Ilustración 3. Diagrama de Gantt - Seguimiento.....	14
Ilustración 4. Arquitectura Cuda.....	16
Ilustración 5. Estructura de una tarjeta gráfica con arquitectura cuda.....	17
Ilustración 6. Estructura de un SM.....	17
Ilustración 7. Ejemplo de código paralelo y código secuencial.....	18
Ilustración 8. Ejecución del ejemplo de código paralelo y código secuencial....	18
Ilustración 9. Jerarquía grid-bloque-hilo.....	19
Ilustración 10. Asignación grid-bloque-hilo gráfica-SM-SP.....	19
Ilustración 11. Jerarquía de memoria.....	20
Ilustración 12. Diagrama de flujo.....	23
Ilustración 13. Compatibilidad versión CUDA - Capacidad de cómputo.....	24
Ilustración 14. Compatibilidad versión CUDA Toolkit - Drivers de Sistemas Operativos.....	25
Ilustración 15. Instalación - Opción de las herramientas de desarrollo de aplicaciones con C++.....	25
Ilustración 16. Instalación - Ejecutar instalador CUDA.....	25
Ilustración 17. Creación proyecto CUDA.....	26
Ilustración 19. Código de la prueba1.....	28
Ilustración 20. Ejecución de la prueba1.....	28
Ilustración 21. Código de la prueba2.....	29
Ilustración 22. Ejecución de la prueba2.....	29
Ilustración 23. Código de la prueba3.....	30
Ilustración 24. Ejecución de la prueba3.....	30
Ilustración 25. Código de la prueba4.....	30
Ilustración 26. Ejecución de la prueba4.....	31
Ilustración 27. Gráfica de la prueba4.....	31
Ilustración 28. Código del sort proporcionado por la librería de C++.....	32
Ilustración 29. Tiempos de Sort de librería de C++.....	32
Ilustración 30. Tiempos de mi implementación.....	33
Ilustración 31. Comparación gráficas.....	33

1 Introducción

Con el avance continuo de los ordenadores y el interés en el desarrollo de aplicaciones cada vez más complejas que requieren cada vez más recursos, la velocidad y la capacidad de procesamiento se han convertido en elementos cruciales para abordar un amplio rango de tareas y desafíos computacionales.

Algunos ejemplos de estas tareas son:

- La creación de sistemas operativos basados en ventanas.
- Aplicaciones de investigación científica que realizan simulaciones complejas en campos como la física, la química, la biología.
- Análisis de datos y la minería de datos, donde se usan algoritmos de procesamiento y transformación de datos.
- Creación de gráficos y procesamiento de imágenes para industrias como las de los videojuegos. [4]

Por esta razón se popularizaron las GPUs (graphics processing unit), ya que son coprocesadores dedicados al procesamiento de gráficos y otras operaciones que reducen la carga de trabajo a la CPU gracias al procesamiento en paralelo.

En el campo del procesado de imágenes tienen cierto papel ciertos algoritmos de ordenación. El que nos interesa en este caso y del que tratará este trabajo es el Counting Sort. Algoritmo de números integrales que ordena en base a las ocurrencias de cada valor dentro del rango de valores [5].

En este trabajo implementaremos dicho algoritmo de forma paralela en Cuda/C para una gráfica, analizando su funcionamiento y las características que le proporciona dicho funcionamiento.

2 Objetivos

El objetivo principal de este trabajo será programar una implementación del algoritmo Counting Sort utilizando el IDE de Microsoft Visual Studio y la arquitectura CUDA.

Para conseguir esto tendremos los siguientes objetivos más específicos:

- **Idear un diseño del algoritmo eficaz:** el programa debe ser capaz de ordenar cualquier array de números positivos de forma paralela.
- **Lograr un código eficiente:** el programa debe de estar optimizado de manera que la velocidad de respuesta y los recursos utilizados sean viables para que sea posible la incorporación del programa en un hipotético proyecto software que requiera de los servicios de ordenamiento de números que proporciona.
- **Mantenibilidad:** el código debe cumplir con los convenios de programación que implican legibilidad, claridad.
- **Manejo de errores:** el código debe controlar e informar sobre cualquier error que pueda surgir durante el proceso de ordenamiento. Especialmente en el caso de que el error sea causado por la falta de memoria para ordenar arrays de tamaños muy elevados.
- **Realización de pruebas:** someter el código a diferentes pruebas que verifiquen su funcionamiento en función del aumento del tamaño del array y rango de valores.
- **Análisis de resultados:** sacar conclusiones de los resultados obtenidos en las pruebas mediante gráficas que representen el tiempo de ejecución en función del tamaño de array de entrada o del tamaño del rango de valores del array.

3 Solución propuesta

Antes de entrar en detalles del diseño paralelo implementado en este trabajo es necesario entender cómo realiza el algoritmo Counting Sort genérico el ordenamiento de un array de números.

3.1 Funcionamiento del algoritmo Counting Sort

Este algoritmo se compone de 2 fases, la fase de conteo y la fase de creación del array ordenado.

En la fase de conteo se hace un conteo de todos los valores dentro del rango de valores del array, es decir, cuenta las ocurrencias de cada valor (creando una lista de ocurrencias).

En la fase de creación del array ordenado se crea el array ordenado en base a esa lista de ocurrencias previamente obtenida.

A partir de este punto utilizaremos las siguientes definiciones:

- **Input:** array de entrada a ordenar.
- **MinMax:** array que representa el rango de valores, siendo el primer elemento el elemento mínimo y el último el elemento máximo.
- **Ocurrencias:** array en el que se almacena el número de veces que aparece cada valor de MinMax en el array de entrada Input.
- **Sorted:** array ordenado resultante.
- **SumaEscalera:** array resultado de hacer una suma acumulativa en Ocurrencias.
- **IndicesNuevos:** array que almacena las posiciones en el array Sorted de cada número del array Input.

Pongamos un ejemplo:

3.1.1 Fase de conteo

Input:

10	2	8	4	6	1	4	2	7
----	---	---	---	---	---	---	---	---

MinMax:

1	2	3	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	----

Ocurrencias:

1	2	0	2	0	1	1	1	0	1
---	---	---	---	---	---	---	---	---	---

Sorted:

-	-	-	-	-	-	-	-	-	-
---	---	---	---	---	---	---	---	---	---

3.1.2 Fase de generación de el array ordenado

Hay dos métodos para generar el array ordenado:

Método1:

Este método consiste en recorrer el array *MinMax* (array del rango de valores) por orden, del mínimo al máximo, y añadir cada valor al final del array sorted tantas veces como se haya contado que esté ese valor en el array de entrada (información que está en el array *Ocurrencias*).

-1ª Iteración

Sorted: - - - - - - - - -

El primer valor de *MinMax* es 1 y vemos en ocurrencias que está una vez en el array de entrada, así que lo añadimos a sorted una vez:

MinMax: 1 2 3 4 5 6 7 8 9 10
Ocurrencias: 1 2 0 2 0 1 1 1 0 1
Sorted: 1 - - - - - - - -

El segundo valor es el 2 y vemos en ocurrencias que está dos veces en el array de entrada, así que lo añadimos a sorted dos veces:

MinMax: 1 2 3 4 5 6 7 8 9 10
Ocurrencias: 1 2 0 2 0 1 1 1 0 1
Sorted: 1 2 2 - - - - - -

Así hasta recorrer todo *MinMax*, quedando el array ordenado.

Sorted: 1 2 2 4 4 6 7 8 10

Método2:

Para este método necesitamos calcular dos arrays.

El array *SumaEscalera*, que es el resultado de hacer una suma acumulativa en *Ocurrencias*.

Ocurrencias:	1	2	0	2	0	1	1	1	0	1
SumaEscalera:	1	3	3	5	5	6	7	8	8	9

Este método sigue los siguientes pasos:

- Coger un valor de *Input*.
- Buscar ese valor en el array *MinMax*, quedándose con el índice en donde se encuentre (al que denominaremos *i*).
- Acceder a ese índice en el array *SumaEscalera*.
- Obtener el valor que se encuentre en ese índice de *SumaEscalera*.
- Restarle 1 a ese valor, y el resultado será el índice en *Sorted*. El valor en *SumaEscalera* queda actualizado para la siguiente vez que se acceda a esa posición, que es en el caso de que se repita el número.

1ª Iteración:

Input:	10	2	8	4	6	1	4	2	7
	0	1	2	3	4	5	6	7	8
MinMax:	1	2	3	4	5	6	7	8	9
	0	1	2	3	4	5	6	7	8
SumaEscalera:	1	3	3	5	5	6	7	8	8
	0	1	2	3	4	5	6	7	8
Sorted:	-	-	-	-	-	-	-	-	10
	0	1	2	3	4	5	6	7	8

2ª iteración:

Input:	10	2	8	4	6	1	4	2	7
	0	1	2	3	4	5	6	7	8
MinMax:	1	2	3	4	5	6	7	8	9
	0	1	2	3	4	5	6	7	8
SumaEscalera:	1	3	3	5	5	6	7	8	8
	0	1	2	3	4	5	6	7	8
Sorted:	-	2	-	-	-	-	-	-	10
	0	1	2	3	4	5	6	7	8

Valor actualizado

3ª Iteración:

Input:	<u>10</u>	<u>2</u>	<u>8</u>	<u>4</u>	<u>6</u>	<u>1</u>	<u>4</u>	<u>2</u>	<u>7</u>	
	0	1	2	3	4	5	6	7	8	
MinMax:	<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>	<u>5</u>	<u>6</u>	<u>7</u>	<u>8</u>	<u>9</u>	<u>10</u>
	0	1	2	3	4	5	6	7	8	9
SumaEscalera:	<u>1</u>	<u>2</u>	<u>3</u>	<u>5</u>	<u>5</u>	<u>6</u>	<u>7</u>	<u>8</u>	<u>8</u>	<u>8</u>
	0	1	2	3	4	5	6	7	8	9
Sorted:	<u>-</u>	<u>2</u>	<u>-</u>	<u>-</u>	<u>-</u>	<u>-</u>	<u>-</u>	<u>8</u>	<u>10</u>	
	0	1	2	3	4	5	6	7	8	

La diferencia de este método con el anterior, es que al acceder al número a colocar en el array *Input*, y después ver en qué posición debe ir en *Sorted*, tenemos la posibilidad de crear un array (al que denomino *IndicesNuevos*) que almacene la posición inicial de cada valor (su índice en *Input*) y su posición final tras ordenar (su índice en *Sorted*). Siendo este array interesante para posibles aplicaciones que quisieran utilizar el algoritmo Counting Sort.

Input:	10	2	8	4	6	1	4	2	7
	0	1	2	3	4	5	6	7	8
Sorted:	1	2	2	4	4	6	7	8	10
	0	1	2	3	4	5	6	7	8
IndicesNuevos:	8	2	7	4	5	0	3	1	7
	0	1	2	3	4	5	6	7	8

3.2 Diseño paralelo propuesto del algoritmo Counting Sort

Ahora procederé a explicar como funciona el algoritmo diseñado en paralelo, ayudándome del mismo ejemplo anterior.

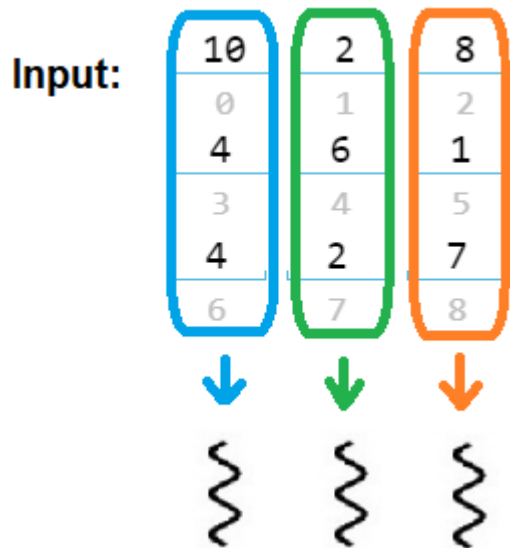
3.2.1 Fase de conteo

Input:	10	2	8	4	6	1	4	2	7
---------------	----	---	---	---	---	---	---	---	---

En este ejemplo ejecutaremos el ordenamiento siendo el número de hilos usados en total igual a 3 ($n_{\text{HilosTotales}} = 3$).

MinMax:	1	2	3	4	5	6	7	8	9	10
----------------	---	---	---	---	---	---	---	---	---	----

El array *Input* se dividirá entre el número de hilos utilizados, haciendo que cada hilo se ocupe de unas posiciones en concreto del array de entrada.



Cada hilo tendrá su propio array *Ocurrencias*.

MinMax:	1	2	3	4	5	6	7	8	9	10
Ocurrencias1:	0	0	0	2	0	0	0	0	0	1
	0	1	2	3	4	5	6	7	8	9
Ocurrencias2:	0	2	0	0	0	1	0	0	0	0
	0	1	2	3	4	5	6	7	8	9
Ocurrencias3:	1	0	0	0	0	0	1	1	0	0
	0	1	2	3	4	5	6	7	8	9

Al finalizar cada hilo, tendremos las ocurrencias de cada número que haya contado cada hilo, por lo que sumaremos los valores que compartan índice en los arrays *Ocurrencias*, para obtener las ocurrencias totales de cada valor.

Los índices a sumar se repartirán entre todos los hilos.

MinMax:	1	2	3	4	5	6	7	8	9	10
Ocurrencias1:	0 0	0 1	0 2	2 3	0 4	0 5	0 6	0 7	0 8	1 9
Ocurrencias2:	0 0	2 1	0 2	0 3	0 4	1 5	0 6	0 7	0 8	0 9
Ocurrencias3:	1 0	0 1	0 2	0 3	0 4	0 5	1 6	1 7	0 8	0 9
OcurrenciasTotales:	1 0	2 1	0 2	2 3	0 4	1 5	1 6	1 7	0 8	1 9
	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓
	⋈	⋈	⋈	⋈	⋈	⋈	⋈	⋈	⋈	⋈

3.2.2 Fase de generar array ordenado

Método1:

Teniendo los arrays *OcurrenciasTotales* y *SumaEscalera*.

Los hilos se reparten los valores del array *MinMax* que van a meter en *Sorted*.

MinMax:	1	2	3	4	5	6	7	8	9	10
OcurrenciasTotales:	1 0	2 1	0 2	2 3	0 4	1 5	1 6	1 7	0 8	1 9
SumaEscalera:	1 0	3 1	3 2	5 3	5 4	6 5	7 6	8 7	8 8	9 9
	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓
	⋈	⋈	⋈	⋈	⋈	⋈	⋈	⋈	⋈	⋈

Los datos necesarios para proceder los tienen los arrays *OcurrenciasTotales* y *SumaEscalera*.

- El valor en *OcurrenciasTotales* la cantidad de veces que insertan el valor en *Sorted*.
- (El valor de *SumaEscalera*) - (el valor de *OcurrenciasTotales*) → es la primera posición en la que se inserta el valor.
- El valor de *SumaEscalera* - 1 es la última posición en la que se inserta el valor.

Por ejemplo:

Al hilo azul le corresponde el valor 1.

El valor correspondiente en *SumaEscalera* es 1. El valor correspondiente en *OcurrenciasTotales* es 1. El resultado de $1-1$ es 0 → Primera posición = 0.

La posición final es igual a el valor en *SumaEscalera* -1 → $1-1=0$.

El hilo azul insertará en *Sorted* desde el índice 0 hasta el 0.

Al hilo verde le corresponde el valor 2.

El valor correspondiente en *SumaEscalera* es 3. El valor correspondiente en *OcurrenciasTotales* es 2. El resultado de $3-2$ es 1 → Primera posición = 1.

La posición final es igual a el valor en *SumaEscalera* -1 → $3-1=2$.

El hilo azul insertará en *Sorted* desde el índice 1 hasta el 2.

Sorted:

1	2	2	4	4	6	7	8	10
0	1	2	3	4	5	6	7	8

Método2:

Este método es igual que en la explicación del algoritmo genérico de el apartado 3.1.2. La única diferencia es que para que sea paralelo, los hilos se reparten los elementos de Input a recolocar, repartiéndoselos de la siguiente manera:

										
										
Input:	10	2	8	4	6	1	4	2	7	
	0	1	2	3	4	5	6	7	8	
MinMax:	1	2	3	4	5	6	7	8	9	10
	0	1	2	3	4	5	6	7	8	9
SumaEscalera:	1	3	3	5	5	6	7	8	8	9
	0	1	2	3	4	5	6	7	8	9

Quedando la primera colocación de cada hilo de la siguiente manera:

										
										
Input:	10 0	2 1	8 2	4 3	6 4	1 5	4 6	2 7	7 8	
MinMax:	1 0	2 1	3 2	4 3	5 4	6 5	7 6	8 7	9 8	10 9
SumaEscalera:	1 0	3 1	3 2	5 3	5 4	6 5	7 6	8 7	8 8	9 9
Sorted:	- 0	- 1	2 2	- 3	- 4	- 5	- 6	8 7	10 8	

El problema que puede surgir al realizar este método en paralelo es que dos hilos intenten colocar a la vez el mismo valor y los dos hilos intenten insertarlo en la misma posición de *Sorted*, debido a que ha surgido una lectura sucia.

La "lectura sucia" es una situación cuando un hilo accede a una ubicación de memoria que está siendo modificada por otro hilo al mismo tiempo.

Es decir, un hilo lee un valor que está siendo modificado por otro hilo, pero hace la lectura antes de que el otro hilo acabe la modificación, haciendo que el valor leído esté desactualizado.

En nuestro caso, una lectura sucia puede provocar que los dos hilos quieran colocar un valor en la misma posición de *Sorted*.

Veamos un ejemplo:

Supongamos esta situación en la que dos hilos distintos tienen que colocar el mismo valor.



Input:	<u>10</u>	<u>2</u>	<u>8</u>	<u>4</u>	<u>6</u>	<u>1</u>	<u>4</u>	<u>2</u>	<u>7</u>	
	0	1	2	3	4	5	6	7	8	
MinMax:	<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>	<u>5</u>	<u>6</u>	<u>7</u>	<u>8</u>	<u>9</u>	<u>10</u>
	0	1	2	3	4	5	6	7	8	9
SumaEscalera:	<u>1</u>	<u>3</u>	<u>3</u>	<u>5</u>	<u>5</u>	<u>6</u>	<u>7</u>	<u>8</u>	<u>8</u>	<u>9</u>
	0	1	2	3	4	5	6	7	8	9
Sorted:	<u>-</u>	<u>-</u>	<u>-</u>	<u>-</u>	<u>-</u>	<u>-</u>	<u>-</u>	<u>-</u>	<u>-</u>	
	0	1	2	3	4	5	6	7	8	

Se podría dar el caso de que los dos accediesen al valor correspondiente en *SumaEscalera* sin que fuese actualizado aun por el otro hilo, y se produjese una lectura sucia.

Diagram illustrating a race condition between two threads (blue and orange) accessing a shared array.

Input:

10	2	8	4	6	1	4	2	7
0	1	2	3	4	5	6	7	8

MinMax:

1	2	3	4	5	6	7	8	9	10
0	1	2	3	4	5	6	7	8	9

SumaEscalera:

1	3	3	5	5	6	7	8	8	9
0	1	2	3	4	5	6	7	8	9

Sorted:

-	-	-	-	-	-	-	-	-
0	1	2	3	4	5	6	7	8

The diagram shows the state of the arrays after the blue thread's update. The orange thread is still at index 7, pointing to the value 2 in the Input array. The blue thread is at index 1, pointing to the value 2 in the Input array. The MinMax array shows the state after the blue thread's update, with index 1 containing 2 and index 0 containing 1. The SumaEscalera array shows the state after the blue thread's update, with index 1 containing 3 and index 0 containing 1. The Sorted array shows the state after the blue thread's update, with index 1 containing - and index 0 containing 0.

La solución para que no suceda esto es acceder y modificar ese valor en *SumaEscalera* de con una acción atómica.

Las instrucciones atómicas representan operaciones que se realizan como una única acción indivisible en un procesador o en una arquitectura paralela, sin posibilidad de interrupciones o entremezclamientos con otras operaciones llevadas a cabo por hilos o procesadores simultáneos, siendo una solución para las lecturas sucias.

En este caso como tenemos que decrementar el valor después de usarlo, por lo que usaremos `atomicSub(int* num, int valor)`. Y así cuando el segundo acceda al valor de

SumaEscalera, ya habrá sido decrementado y así colocarán correctamente los dos valores en donde corresponde del array *Sorted*.

4 Metodología Usada

Tras realizar el planteamiento inicial del proyecto, se tomó la decisión de que la metodología usada sería el Desarrollo basado en pruebas (TTD). En esta metodología las pruebas pasan a ser la parte principal del desarrollo.

El TTD pasa por varias fases:

- **Fase Roja:** Primero se escriben las pruebas que verifiquen que las funcionalidades se vayan a cumplir. Estas pruebas al principio deben fallar, ya que aun no se ha desarrollado el software que haga que salga exitosa.
- **Fase Verde:** tras escribir las pruebas, se empieza a escribir la implementación que haga que la prueba salga exitosa.
- **Refactorización:** tras escribir la implementación, se realizan las pruebas para comprobar que la implementación es buena y finalmente, después de que la implementación pase todas las pruebas, se refactoriza y simplifica el código [6].

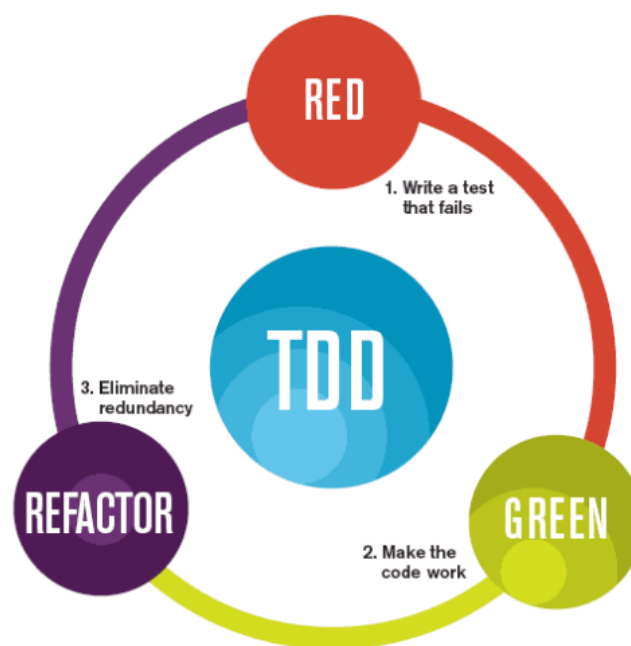


Ilustración 1. Metodología TTD.

5 Planificación y seguimiento

A continuación se presentan dos diagramas de Gantt. Uno correspondiente a la planificación, en el cual se representa la duración estimada de las tareas necesarias para la realización del TFG; y otro, correspondiente al seguimiento, en el cual se representa la duración real de las tareas.

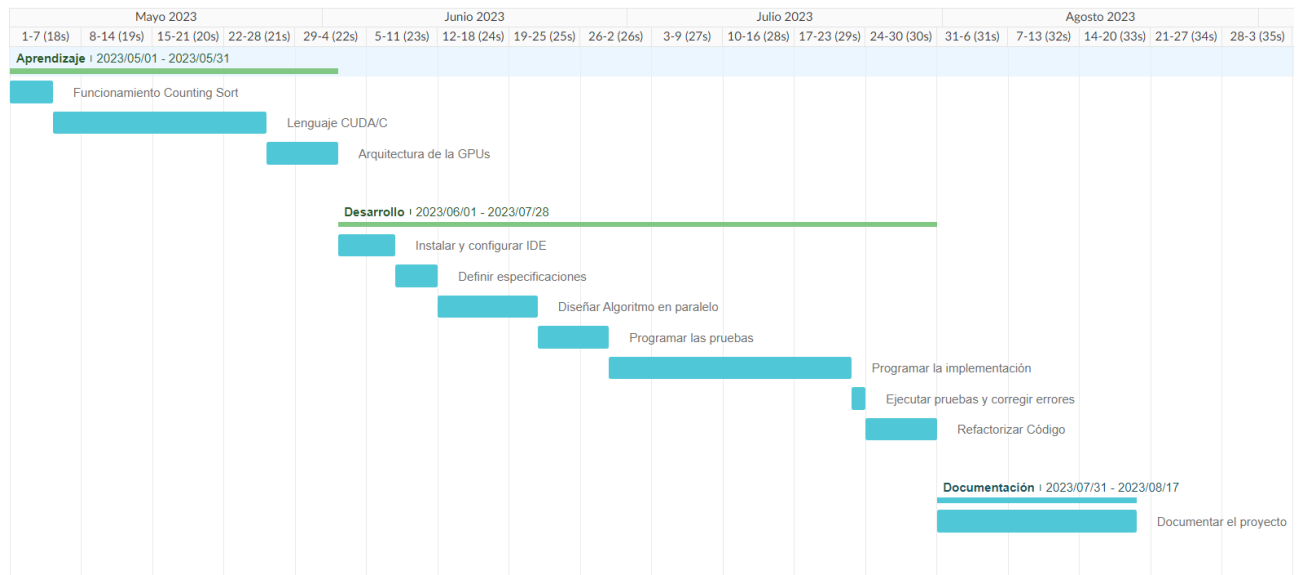


Ilustración 2. Diagrama de Gantt - Planificación inicial.

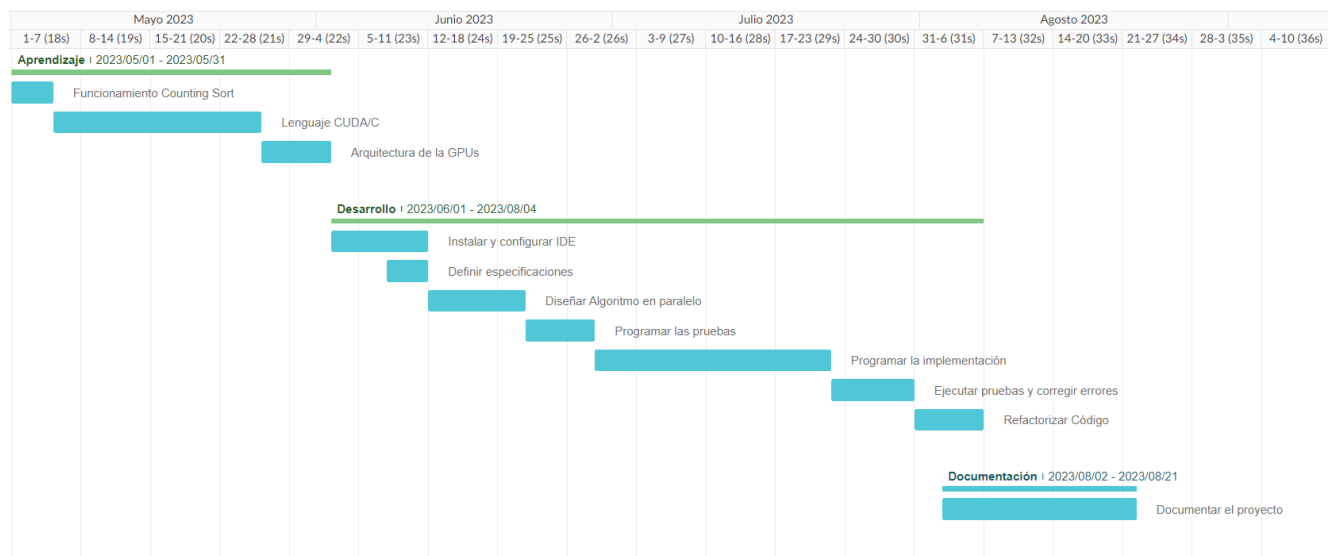


Ilustración 3. Diagrama de Gantt - Seguimiento.

Como se puede observar en los diagramas de Gantt, no hubo ningún problema en la fase de aprendizaje de los conceptos necesarios para comenzar con el desarrollo, cumpliendo con los plazos previstos en la planificación.

Al comenzar el desarrollo, hubo problemas con la instalación y configuración del IDE, alargándose la tarea durante varios días. Debido a esto se compaginó esta tarea con la de definir las especificaciones del programa, para así no retrasar el comienzo del diseño del algoritmo en paralelo y tareas posteriores.

No surgieron más imprevistos hasta la ejecución de las pruebas y corrección de errores, tardando más de lo previsto en la corrección de errores, lo cual retrasó el comienzo de la refactorización de código.

Se tomó la decisión de comenzar con la documentación del proyecto mientras se terminaba de refactorizar el código para evitar retrasar más la fecha de finalización del proyecto.

6 Arquitectura

Este proyecto está diseñado para ser ejecutado en un computador con una tarjeta gráfica con Arquitectura Cuda y con soporte para lenguajes de alto nivel como C y C++ .

La arquitectura CUDA está diseñada para cálculo paralelo de propósito general ofreciendo un nuevo repertorio de instrucciones y un nuevo modelo de programación paralela siendo constituidas por cientos de núcleos que pueden procesar de manera concurrente miles de hilos de ejecución [7].

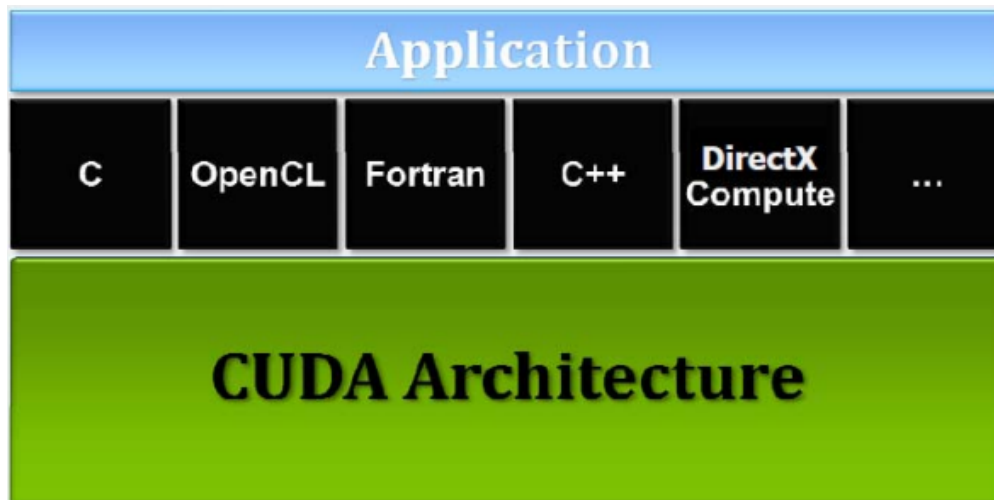


Ilustración 4. Arquitectura Cuda.

En la arquitectura clásica de una gráfica podemos encontrar dos tipos de procesadores, los procesadores de vértices y los procesadores de fragmentos, teniendo estos tareas distintas y conjuntos de instrucciones diferentes, presentando esto diferentes problemas. Siendo así la evolución natural de la arquitectura de las gpus a la arquitectura cuda, donde los núcleos requieren del mismo repertorio de instrucciones y recursos [7].

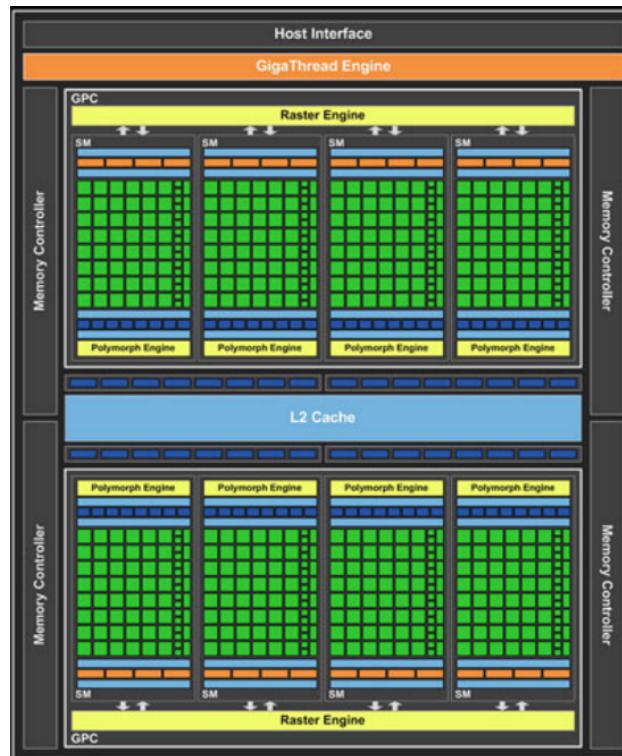


Ilustración 5. Estructura de una tarjeta gráfica con arquitectura cuda.

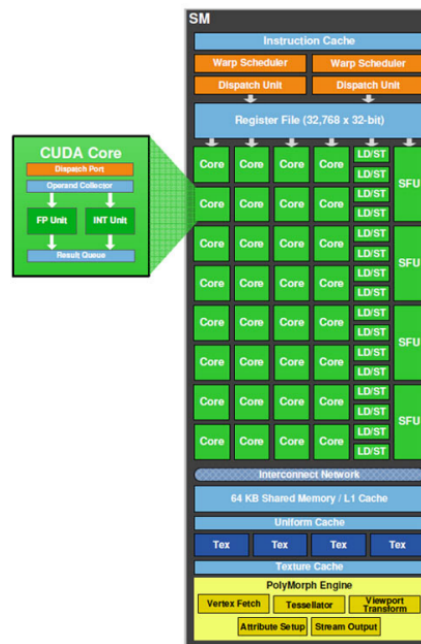


Ilustración 6. Estructura de un SM.

En las imágenes anteriores se muestra la estructura de una tarjeta gráfica con arquitectura cuda. Estas están formadas por unidades de ejecución interconectadas entre sí por una zona de memoria común conocidas como Streaming Multiprocessors (SM).

Cada SM está compuesto a su vez por unos núcleos de cómputo, que son los encargados de ejecutar las instrucciones, conocidos como Streaming Processors (SP). En la figura de ejemplo vemos que hay 8 SM y 32 SP por cada SM [7].

El lenguaje CUDA dota al programador de una abstracción que le permite poder programar de manera secuencial y en ese código secuencial llamar a una función que se ejecuta en paralelo conocida como kernel.

```

1  #include "cuda_runtime.h"
2  #include "cuda_runtime_api.h"
3  #include "device_launch_parameters.h"
4  #include <stdio.h>
5
6  __global__ void kernel() {
7      printf("Print ejecutado en la GPU por el hilo %d del bloque %d \n", threadIdx.x, blockIdx.x);
8  }
9
10 int main(int argc, char* argv[]) {
11
12     int numBloques = 2;
13     int numHilos = 3;
14     printf("Print ejecutado en la CPU\n");
15     kernel << <numBloques, numHilos >> > ();
16     cudaDeviceSynchronize();
17 }

```

Código Paralelo

Código Secuencial

Ilustración 7. Ejemplo de código paralelo y código secuencial.

```

Print ejecutado en la CPU
Print ejecutado en la GPU por el hilo 0 del bloque 1
Print ejecutado en la GPU por el hilo 1 del bloque 1
Print ejecutado en la GPU por el hilo 2 del bloque 1
Print ejecutado en la GPU por el hilo 0 del bloque 0
Print ejecutado en la GPU por el hilo 1 del bloque 0
Print ejecutado en la GPU por el hilo 2 del bloque 0

```

Ilustración 8. Ejecución del ejemplo de código paralelo y código secuencial.

El kernel se ejecuta de forma paralela en la GPU como un conjunto de hilos (threads), siendo agrupados los hilos en lo que se conoce como bloques (blocks), y siendo agrupados éstos a su vez en lo que se conoce como grid. Un aspecto a concretar es que al lanzar un grid a una gráfica, los bloques a ejecutar se asignan a los SM, los hilos de cada bloque se asignan a los SP [7].

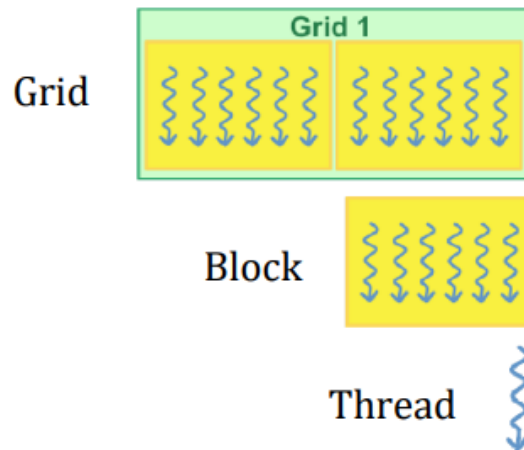


Ilustración 9. Jerarquía grid-bloque-hilo.

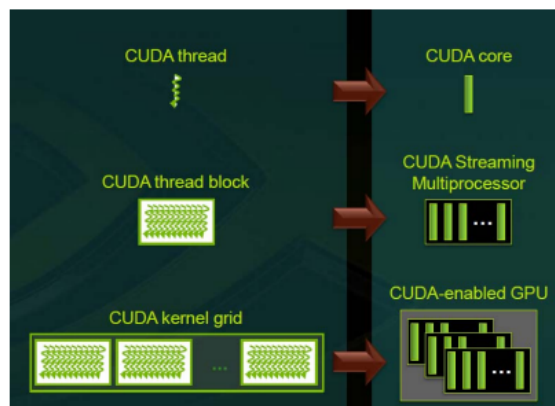


Ilustración 10. Asignación grid-bloque-hilo gráfica-SM-SP.

En cuanto a la jerarquía de memoria, cada hilo tiene una zona de memoria privada y cada bloque tiene una zona de memoria compartida para todos los hilos de ese bloque. También hay una zona de memoria global, siendo esta la única memoria accesible desde el host y siendo compartida entre todos los hilos del device.

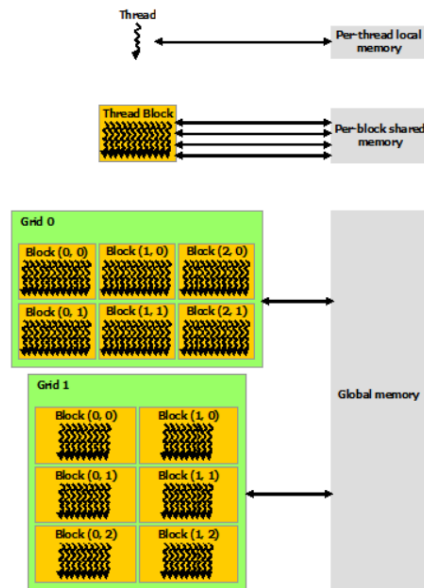


Ilustración 11. Jerarquía de memoria.

7 Tecnologías e integración de productos de terceros

Este apartado está dedicado a la mención y breve explicación de las tecnologías y productos que han sido necesarios para el desarrollo de la implementación del algoritmo Counting Sort en paralelo.

- CUDA-C (Compute Unified Architecture -C): extensión del lenguaje de programación C desarrollada por NVIDIA para aprovechar la potencia de las GPUs en aplicaciones de gran paralelismo y alto cómputo.
- CUDA Toolkit: conjunto de herramientas y recursos desarrollado por NVIDIA para facilitar la programación de aplicaciones, entre las que se encuentra el compilador, depurador y librerías de CUDA.
- CUDA SDK: conjunto de herramientas y recursos desarrollado por NVIDIA para facilitar la programación de aplicaciones, entre las que se encuentran plantillas de proyectos, ejemplo de códigos.
- Microsoft Visual Studio: conocido IDE desarrollado por Microsoft compatible con múltiples lenguajes de programación. Este nos proporciona principalmente el compilador de C entre otras herramientas para el proyecto.
- GitHub: plataforma en línea que proporciona servicios para el desarrollo colaborativo de software utilizando el sistema de control de versiones distribuido Git.

8 Especificación y análisis de requisitos

Los requisitos del programa son simples:

- Dado un array desordenado de números integrales, devolver ese mismo array ordenado mediante el algoritmo Counting Sort implementado de forma paralela.
- Debe devolver otros arrays creados en el proceso que pueden ser interesantes en el uso del algoritmo en otras aplicaciones mayores. Estos arrays son los anteriormente explicados: *indicesNuevos* y *ocurrencias*.
- Este programa no tiene interacción con el usuario ya que no está pensado para usarse por sí solo, está pensado para coger el código y añadirlo al proyecto mayor que requiera de los servicios del algoritmo Counting Sort.
- El código debe ser modular. Debe tener una función principal llamada `CountingSortParalelo(int* input, int inputSize, int numBloques, int numHilos, char metodoCreaSorted)` que se encargará de realizar todo el ordenamiento llamando a subfunciones en el proceso.
- El programa debe no controlar todos los errores que se puedan producir a la hora de realizar la ordenación, controlando el espacio de memoria que utiliza para evitar fallos en caso de que el array de entrada sea de proporciones demasiado grandes para la memoria que se dispone.

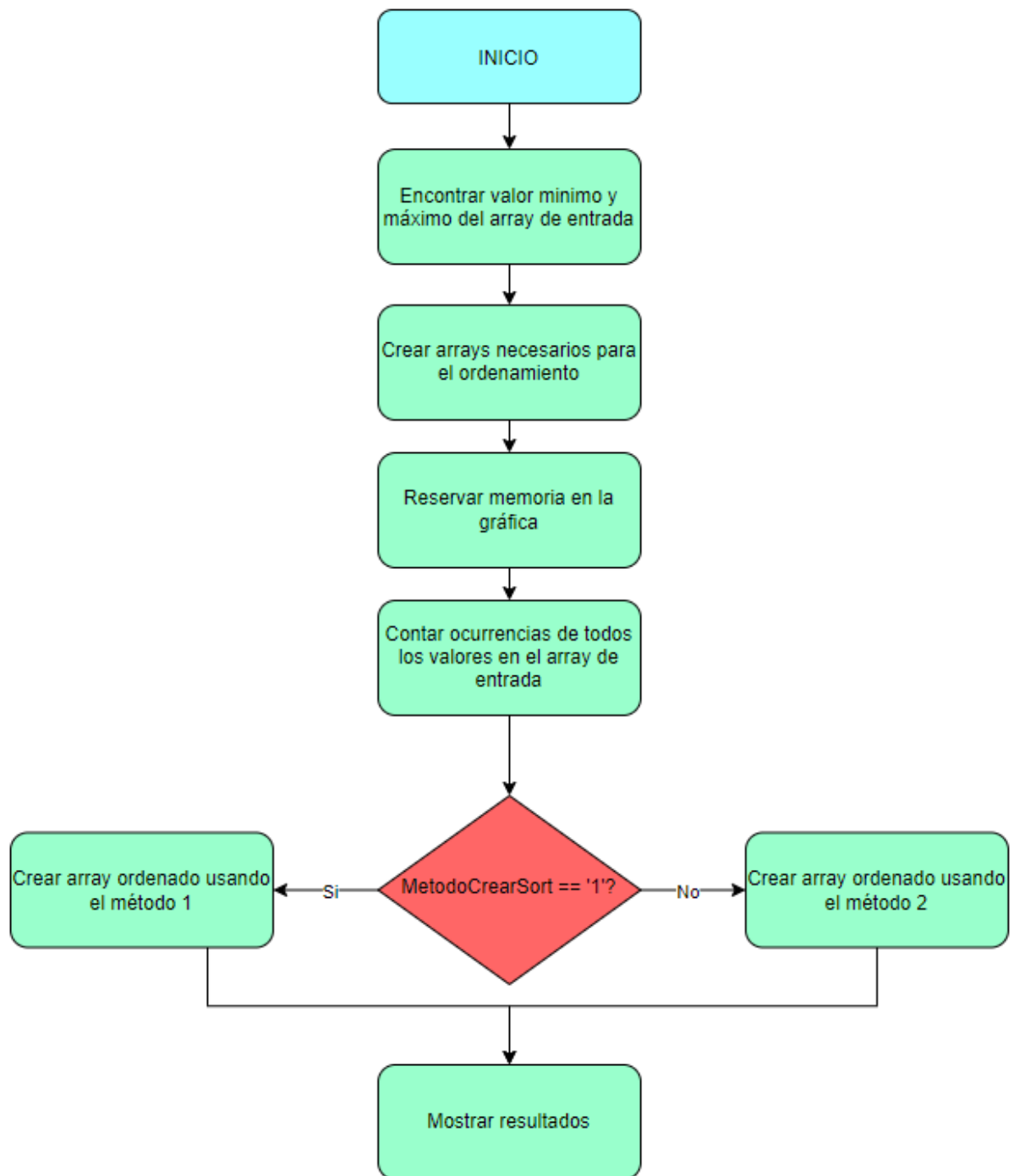


Ilustración 12. Diagrama de flujo.

9 Manual de usuario

9.1 Requisitos Mínimos

- Tarjeta gráfica NVIDIA con la característica “CUDA-enabled”. Dependiendo de la versión de CUDA que vayamos a utilizar necesitaremos una tarjeta con una mínima capacidad de cómputo.
- Driver de dispositivo. El driver también debe ser compatible con la versión de CUDA.
- Compilador de C/C++.
- Software de CUDA disponible en la página web de NVIDIA

Versión de CUDA	Capacidad de cómputo soportada
6.5	1.0 – 5.3
7.5	2.0 – 5.3
8.0	2.0 – 6.2
9.2	3.0 – 7.2
10.2	3.0 – 7.5
11.0	5.0 – 8.0
¿?	¿?

Ilustración 13. Compatibilidad versión CUDA - Capacidad de cómputo.

CUDA Toolkit	Linux x86_64 Driver Version	Windows x86_64 Driver Version
CUDA 11.0.194	>= 450.51.05	>= 451.48
CUDA 11.0.171 RC	>= 450.36.06	>= 451.22
CUDA 10.2.89	>= 440.33	>= 441.22
CUDA 10.1 (10.1.105 general release, and updates)	>= 418.39	>= 418.96
CUDA 10.0.130	>= 410.48	>= 411.31
CUDA 9.2 (9.2.148 Update 1)	>= 396.37	>= 398.26
CUDA 9.2 (9.2.88)	>= 396.26	>= 397.44
CUDA 9.1 (9.1.85)	>= 390.46	>= 391.29
CUDA 9.0 (9.0.76)	>= 384.81	>= 385.54
CUDA 8.0 (8.0.61 GA2)	>= 375.26	>= 376.51
CUDA 8.0 (8.0.44)	>= 367.48	>= 369.30
CUDA 7.5 (7.5.16)	>= 352.31	>= 353.66
CUDA 7.0 (7.0.28)	>= 346.46	>= 347.62

9.2 Guía de Instalación

9.2.1 Instalación de Microsoft Visual Studio

- Descargar el instalador de Microsoft Visual Studio disponible en aquí: [instalador de Microsoft Visual Studio](#)
- Ejecutar el instalador y marcar la opción en la instalación de descargar las herramientas para el desarrollo de aplicaciones de escritorio con C++.

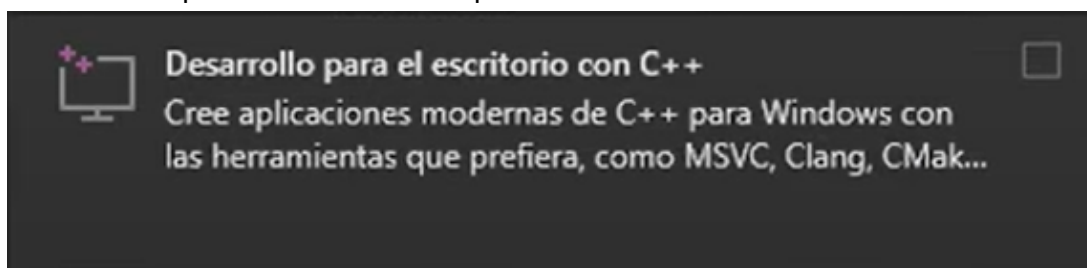


Ilustración 15. Instalación - Opción de las herramientas de desarrollo de aplicaciones con C++.

9.2.2 Instalación de CUDA

- Descargar el instalador de cuda disponible en el siguiente enlace : [instalador de cuda](#).
- Ejecutar el instalador y seguir sus instrucciones.

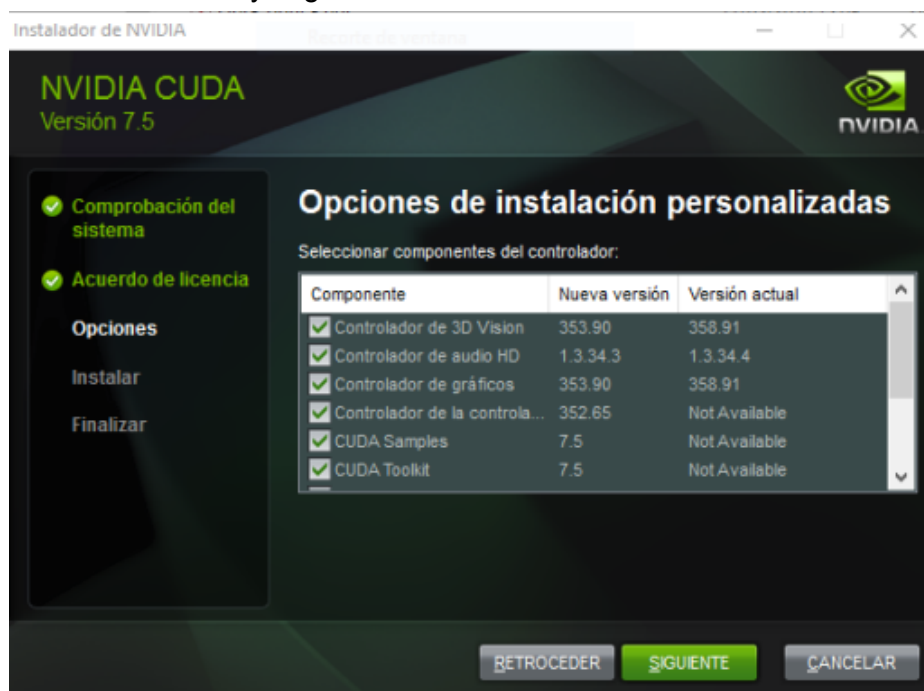


Ilustración 16. Instalación - Ejecutar instalador CUDA.

- Una vez instalado todo, podrás crear un proyecto cuda en el cual añadir el código (en el archivo de extensión .cu) creado por mi disponible en el siguiente enlace:
<https://github.com/SaulCandan/CountingSort-TFG.git>

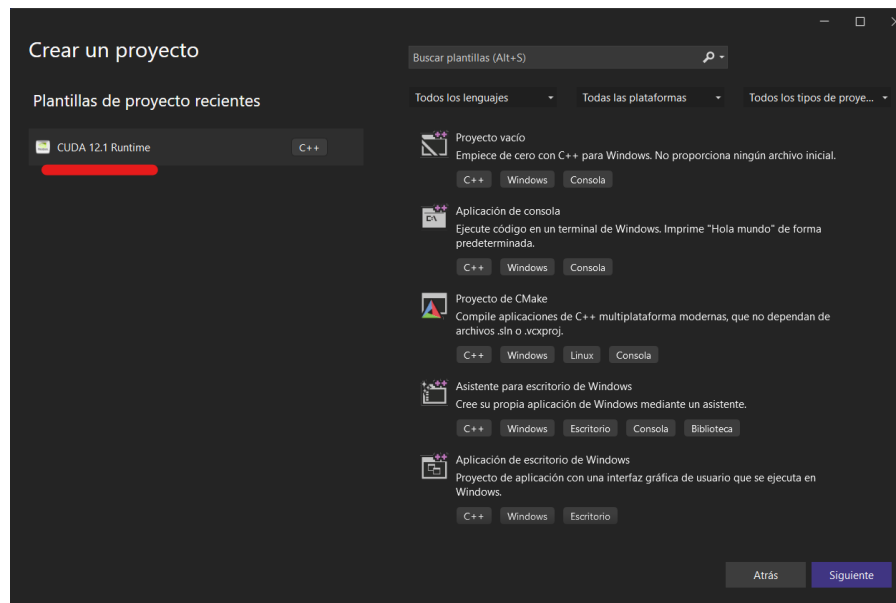


Ilustración 17. Creación proyecto CUDA.

9.3 Manual de Utilización

Para realizar el ordenamiento sobre un array solo tenemos que llamar a la función principal `CountingSortParalelo(int* input, int inputSize, int numBloques, int numHilosPorBloque, char metodoCrearSort)` con los parámetros adecuados que se indican a continuación.

- `input`: array de números a ordenar.
- `size`: tamaño del array `input`.
- `numBloques`: número de bloques utilizados para ordenar.
- `numHilosPorBloque`: número de hilos por bloque utilizados para ordenar.
- `metodoCreaSorted`: char que indica cuál de los métodos para crear el array ordenado utilizar ('1' o '2').

```

int main(int argc, char* argv[]) {

    //PRESENTACION
    presentacion();

    int size = 10;
    int* input = generarArrayAleatorio(size, 1, 65535);
    int numBloques = 1;
    int numHilosPorBloque = 32;
    char metodoCreaSorted = '1';

    CountingSortParalelo(input, size, numBloques, numHilosPorBloque, metodoCreaSorted);

    return 0;
}

```

Ilustración 18. Ejemplo de llamada a la función principal.

10 Pruebas

A continuación mostraré una serie de ejecuciones que prueben tanto la eficacia, eficiencia y escalabilidad del programa.

- **Prueba1:** en esta prueba llamamos a la función de ordenación con un array de tamaño pequeño (10) para poder ver los resultados y corroborar que son correctos.

```
void prueba1() {  
    int size = 10;  
    int* input = generarArrayAleatorio(size, 1, size);  
  
    CountingSortParalelo(input, size, 1, 3, '1');  
    CountingSortParalelo(input, size, 1, 3, '2');  
}
```

Ilustración 19. Código de la prueba1.

```
Usando la grafica: NVIDIA GeForce GTX 1050  
Maximo de bloques por multiprocesador : 32  
Maximo de hilos por bloque: 1024  
Maximo de hilos posibles: 2048  
Warp: 32  
Memoria disponible en la GPU: 3541041152 bytes  
  
Testeando... Ordenado Paralelo Correcto  
Bloques,Hilos: 1,3      Tamano array: 10      Valores: 9, [1,9]      Tiempo tardado: 0.731488 milisegundos  
  
¿Quieres ver el array input? (s/n): s  
1, 6, 2, 9, 6, 5, 4, 9, 9, 8,  
  
¿Quieres ver el array sorted? (s/n): s  
1, 2, 4, 5, 6, 6, 8, 9, 9, 9,  
  
¿Quieres ver el array aux? (s/n): s  
1, 1, 0, 1, 1, 2, 0, 1, 3,  
  
Testeando... Ordenado Paralelo Correcto  
Bloques,Hilos: 1,3      Tamano array: 10      Valores: 9, [1,9]      Tiempo tardado: 0.700416 milisegundos  
  
¿Quieres ver el array input? (s/n): s  
1, 6, 2, 9, 6, 5, 4, 9, 9, 8,  
  
¿Quieres ver el array indicesNuevos? (s/n): s  
0, 5, 1, 9, 4, 3, 2, 8, 7, 6,  
  
¿Quieres ver el array sorted? (s/n): s  
1, 2, 4, 5, 6, 6, 8, 9, 9, 9,  
  
¿Quieres ver el array aux? (s/n): s  
1, 1, 0, 1, 1, 2, 0, 1, 3,
```

Ilustración 20. Ejecución de la prueba1.

- **Prueba2:** para esta prueba estarán comentados los prints de salida ya que probaremos el programa con arrays de tamaños muy altos.

```

714 void prueba2() {
715     //100
716     int size100 = 100;
717     int* input100 = generarArrayAleatorio(size100, 1, size100);
718     CountingSortParalelo(input100, size100, 32, 32, '1');
719     //500
720     int size500 = 500;
721     int* input500 = generarArrayAleatorio(size500, 1, size500);
722     CountingSortParalelo(input500, size500, 32, 32, '2');
723     //1000
724     int size1000 = 1000;
725     int* input1000 = generarArrayAleatorio(size1000, 1, size1000);
726     CountingSortParalelo(input1000, size1000, 32, 32, '1');
727     //10000
728     int size10000 = 10000;
729     int* input10000 = generarArrayAleatorio(size10000, 1, size10000);
730     CountingSortParalelo(input10000, size10000, 32, 32, '2');
731     //50000
732     int size50000 = 50000;
733     int* input50000 = generarArrayAleatorio(size50000, 1, size50000);
734     CountingSortParalelo(input50000, size50000, 32, 32, '1');
735     //200000
736     int size200000 = 200000;
737     int* input200000 = generarArrayAleatorio(size200000, 1, size200000);
738     CountingSortParalelo(input200000, size200000, 32, 32, '2');
739     //500000
740     int size500000 = 500000;
741     int* input500000 = generarArrayAleatorio(size500000, 1, size500000);
742     CountingSortParalelo(input500000, size500000, 32, 32, '1');
743 }
744

```

Ilustración 21. Código de la prueba2.

```

Usando la grafica: NVIDIA GeForce GTX 1050
Maximo de bloques por multiprocesador : 32
Maximo de hilos por bloque: 1024
Maximo de hilos posibles: 2048
Warp: 32
Memoria disponible en la GPU: 3541041152 bytes

Testeando... Ordenado Paralelo Correcto
Bloques,Hilos: 32,32 Tamano array: 100 Valores: 100, [1,100] Tiempo tardado: 1.027072 milisegundos

Testeando... Ordenado Paralelo Correcto
Bloques,Hilos: 32,32 Tamano array: 500 Valores: 500, [1,500] Tiempo tardado: 2.646016 milisegundos

Testeando... Ordenado Paralelo Correcto
Bloques,Hilos: 32,32 Tamano array: 1000 Valores: 1000, [1,1000] Tiempo tardado: 5.351424 milisegundos

Testeando... Ordenado Paralelo Correcto
Bloques,Hilos: 32,32 Tamano array: 10000 Valores: 10000, [1,10000] Tiempo tardado: 35.831806 milisegundos

Testeando... Ordenado Paralelo Correcto
Bloques,Hilos: 32,32 Tamano array: 50000 Valores: 50001, [1,50001] Tiempo tardado: 152.836090 milisegundos

Testeando... Ordenado Paralelo Correcto
Bloques,Hilos: 32,32 Tamano array: 200000 Valores: 200001, [1,200001] Tiempo tardado: 470.648834 milisegundos

Testeando... Ordenado Paralelo Correcto
Bloques,Hilos: 32,32 Tamano array: 500000 Valores: 500001, [1,500001] Tiempo tardado: 1168.260620 milisegundos

```

Ilustración 22. Ejecución de la prueba2.

- **Prueba3:** en esta prueba comprobaremos que ante un array tan grande con el que no se pueda realizar el ordenamiento por la limitación de memoria, el programa controla el error y libera la memoria permitiendo seguir con la ejecución y ordenar otros arrays.

```

void prueba3() {
    //Size demasiado grande
    int sizeDemasiadoGrande = 1000000000;
    int* inputDemasiadoGrande = generarArrayAleatorio(sizeDemasiadoGrande, 1, 10000);
    CountingSortParalelo(inputDemasiadoGrande, sizeDemasiadoGrande, 32, 32, '1');

    //Size viable
    int sizeViable = 100;
    int* inputViable = generarArrayAleatorio(sizeViable, 1, 10000);
    CountingSortParalelo(inputViable, sizeViable, 32, 32, '1');
}

```

Ilustración 23. Código de la prueba3.

```

Usando la grafica: NVIDIA GeForce GTX 1050
Maximo de bloques por multiprocesador : 32
Maximo de hilos por bloque: 1024
Maximo de hilos posibles: 2048
Warp: 32
Memoria disponible en la GPU: 3541041152 bytes

cudaMalloc failed! (Array: INPUT)
Se ha producido algun error
Seguramente al reservar memoria debido al gran tamaño de algun array
Reseteando device...

Testeando... Ordenado Paralelo Correcto
Bloques,Hilos: 32,32    Tamaño array: 100    Valores: 9945, [49,9993]    Tiempo tardado: 12.559360 milisegundos

```

Ilustración 24. Ejecución de la prueba3.

- **Prueba4:** en esta prueba comprobaremos ante un tamaño de array fijo de entrada, como varía el tiempo de ejecución en función del número de hilos totales llamados. Incrementando el número de hilos totales hasta el máximo permitido por la gráfica del ordenador en el que se han hecho estas pruebas: 2048 hilos.

```

void prueba4() {
    int size = 50000000;
    int* input = generarArrayAleatorio(size, 1, 65535);
    CountingSortParalelo(input, size, 1, 1, '1');
    CountingSortParalelo(input, size, 1, 2, '1');
    CountingSortParalelo(input, size, 1, 4, '1');
    CountingSortParalelo(input, size, 1, 8, '1');
    CountingSortParalelo(input, size, 1, 16, '1');
    CountingSortParalelo(input, size, 1, 32, '1');
    CountingSortParalelo(input, size, 1, 64, '1');
    CountingSortParalelo(input, size, 1, 128, '1');
    CountingSortParalelo(input, size, 1, 256, '1');
    CountingSortParalelo(input, size, 1, 512, '1');
    CountingSortParalelo(input, size, 1, 1024, '1');
    CountingSortParalelo(input, size, 5, 256, '1');
    CountingSortParalelo(input, size, 6, 256, '1');
    CountingSortParalelo(input, size, 7, 256, '1');
    CountingSortParalelo(input, size, 2, 1024, '1');
}

```

Ilustración 25. Código de la prueba4.

Testeando... Ordenado	Paralelo	Correcto			
Bloques,Hilos: 1,1	Tamano array: 50000000	Valores: 65536, [1,65536]	Tiempo tardado: 147013.609375 milisegundos		
Testeando... Ordenado	Paralelo	Correcto			
Bloques,Hilos: 1,2	Tamano array: 50000000	Valores: 65536, [1,65536]	Tiempo tardado: 103339.296875 milisegundos		
Testeando... Ordenado	Paralelo	Correcto			
Bloques,Hilos: 1,4	Tamano array: 50000000	Valores: 65536, [1,65536]	Tiempo tardado: 53347.925781 milisegundos		
Testeando... Ordenado	Paralelo	Correcto			
Bloques,Hilos: 1,8	Tamano array: 50000000	Valores: 65536, [1,65536]	Tiempo tardado: 27475.117188 milisegundos		
Testeando... Ordenado	Paralelo	Correcto			
Bloques,Hilos: 1,16	Tamano array: 50000000	Valores: 65536, [1,65536]	Tiempo tardado: 14041.342773 milisegundos		
Testeando... Ordenado	Paralelo	Correcto			
Bloques,Hilos: 1,32	Tamano array: 50000000	Valores: 65536, [1,65536]	Tiempo tardado: 7268.877441 milisegundos		
Testeando... Ordenado	Paralelo	Correcto			
Bloques,Hilos: 1,64	Tamano array: 50000000	Valores: 65536, [1,65536]	Tiempo tardado: 3780.925537 milisegundos		
Testeando... Ordenado	Paralelo	Correcto			
Bloques,Hilos: 1,128	Tamano array: 50000000	Valores: 65536, [1,65536]	Tiempo tardado: 2027.302856 milisegundos		
Testeando... Ordenado	Paralelo	Correcto			
Bloques,Hilos: 1,256	Tamano array: 50000000	Valores: 65536, [1,65536]	Tiempo tardado: 1134.816284 milisegundos		
Testeando... Ordenado	Paralelo	Correcto			
Bloques,Hilos: 1,512	Tamano array: 50000000	Valores: 65536, [1,65536]	Tiempo tardado: 712.281067 milisegundos		
Testeando... Ordenado	Paralelo	Correcto			
Bloques,Hilos: 1,1024	Tamano array: 50000000	Valores: 65536, [1,65536]	Tiempo tardado: 598.711304 milisegundos		
Testeando... Ordenado	Paralelo	Correcto			
Bloques,Hilos: 5,256	Tamano array: 50000000	Valores: 65536, [1,65536]	Tiempo tardado: 481.345551 milisegundos		
Testeando... Ordenado	Paralelo	Correcto			
Bloques,Hilos: 6,256	Tamano array: 50000000	Valores: 65536, [1,65536]	Tiempo tardado: 469.166077 milisegundos		
Testeando... Ordenado	Paralelo	Correcto			
Bloques,Hilos: 7,256	Tamano array: 50000000	Valores: 65536, [1,65536]	Tiempo tardado: 459.695099 milisegundos		
Testeando... Ordenado	Paralelo	Correcto			
Bloques,Hilos: 2,1024	Tamano array: 50000000	Valores: 65536, [1,65536]	Tiempo tardado: 514.671631 milisegundos		

Ilustración 26. Ejecución de la prueba4.

Como se puede observar en la gráfica de la Ilustración 24, el tiempo de ejecución es inversamente proporcional al número de hilos utilizados para realizar el ordenamiento. Prueba esto de el correcto funcionamiento de la implementación. También es interesante señalar que el tiempo de ejecución es considerablemente mejorado en los primeros incrementos en el número de hilos, habiendo mucha más mejora por ejemplo cuando pasas de 16 a 32 hilos, que cuando pasas de 1024 a 2048 hilos.

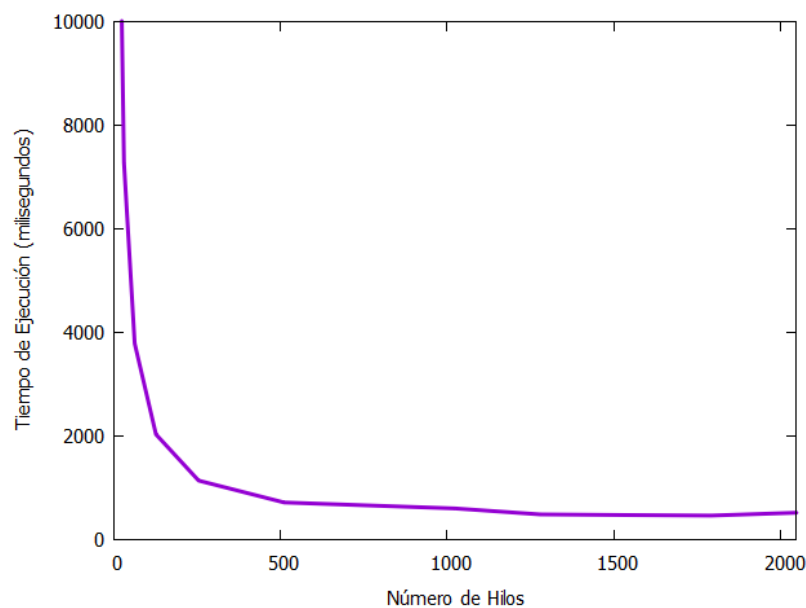


Ilustración 27. Gráfica de la prueba4.

- **Prueba5:** en esta prueba compararemos la eficiencia de el algoritmo implementado en este trabajo con el método de ordenación sort proporcionado por la librería de C++. Haremos 14 ordenamientos probando desde un tamaño de array 10 hasta 40.000.000, aumentando exponencialmente.

El código con el que usamos método de ordenación sort proporcionado por la librería de C++ es el siguiente:

```
#include<iostream>
#include<algorithm>
using namespace std;
void SortLibreriaC (int* input, int size) {

    //INICIAMOS CRONOMETRO TIEMPO
    cudaEvent_t inicio, fin;
    cudaEventCreate(&inicio);
    cudaEventCreate(&fin);
    cudaEventRecord(inicio, 0);

    std::sort(input, input + size); //Sorting array

    //PARAMOS CRONOMETRO
    cudaEventRecord(fin, 0);
    cudaEventSynchronize(fin);
    float tiempo;
    cudaEventElapsedTime(&tiempo, inicio, fin);

    printf("Ordenado Correcto Tamano array: %d\n ", size);
    printf("Tiempo tardado: %f milisegundos\n\n", tiempo);
}
```

Ilustración 28. Código del sort proporcionado por la librería de C++.

Ordenado Correcto Tamano array: 10 Tiempo tardado: 0.014336 milisegundos	Ordenado Correcto Tamano array: 15000000 Tiempo tardado: 5034.146973 milisegundos
Ordenado Correcto Tamano array: 100 Tiempo tardado: 0.037888 milisegundos	Ordenado Correcto Tamano array: 30000000 Tiempo tardado: 12496.169922 milisegundos
Ordenado Correcto Tamano array: 1000 Tiempo tardado: 0.210944 milisegundos	Ordenado Correcto Tamano array: 50000000 Tiempo tardado: 93603.453125 milisegundos
Ordenado Correcto Tamano array: 10000 Tiempo tardado: 3.074048 milisegundos	Ordenado Correcto Tamano array: 80000000 Tiempo tardado: 48607.000000 milisegundos
Ordenado Correcto Tamano array: 50000 Tiempo tardado: 16.569344 milisegundos	Ordenado Correcto Tamano array: 150000000 Tiempo tardado: 55861.277344 milisegundos
Ordenado Correcto Tamano array: 1000000 Tiempo tardado: 397.694977 milisegundos	Ordenado Correcto Tamano array: 250000000 Tiempo tardado: 93622.664062 milisegundos
Ordenado Correcto Tamano array: 5000000 Tiempo tardado: 1654.753296 milisegundos	Ordenado Correcto Tamano array: 400000000 Tiempo tardado: 158688.328125 milisegundos

Ilustración 29. Tiempos de Sort de librería de C++.

Tiempo tardado: 71.705757 milisegundos	Tiempo tardado: 296.759308 milisegundos
Tiempo tardado: 74.900482 milisegundos	Tiempo tardado: 374.618103 milisegundos
Tiempo tardado: 114.990082 milisegundos	Tiempo tardado: 477.369354 milisegundos
Tiempo tardado: 187.142151 milisegundos	Tiempo tardado: 644.046875 milisegundos
Tiempo tardado: 170.479614 milisegundos	Tiempo tardado: 1032.955933 milisegundos
Tiempo tardado: 174.580734 milisegundos	Tiempo tardado: 1583.056885 milisegundos
Tiempo tardado: 244.806656 milisegundos	Tiempo tardado: 2463.405029 milisegundos

Ilustración 30. Tiempos de mi implementación.

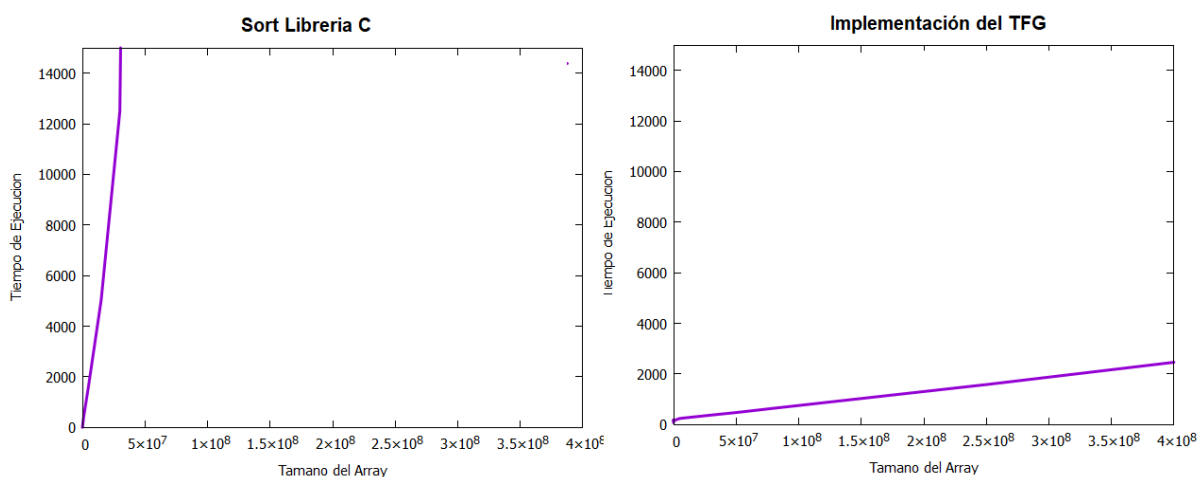


Ilustración 31. Comparación gráficas.

Como podemos observar con la comparación de las gráficas, el sort de la librería de C++ es más eficiente hasta la ordenación del array de tamaño de 100.000 (siendo la diferencia de milisegundos relativamente pequeña).

A partir de las ordenaciones con arrays de tamaño mayor, mi implementación es más eficiente de manera muy diferencial. Habiendo una diferencia de hasta 156.000 milisegundos de tiempo de ejecución en la última ordenación (tamaño del array = 400.000000).

Dados los datos resultantes, podemos sacar en conclusión que las dos implementaciones son eficientes y utilizables hasta cierto límite de tamaño, a partir del cual, es muchísimo más eficiente la implementación realizada en este TFG.

11 Conclusiones y aportaciones

11.1 Principales aportaciones

Teniendo en cuenta los objetivos preestablecidos y las pruebas realizadas, podemos afirmar que se han logrado las siguientes aportaciones:

- Creación de una herramienta eficiente y eficaz de ordenación de arrays de números integrales paralela que implementa el algoritmo Counting Sort.

11.2 Conclusiones

A modo de conclusión, la experiencia general realizando este trabajo ha sido positiva, logrando aprender de forma autodidacta sobre varios campos en los que no tenía ni la suficiente experiencia y ni el conocimiento. Entre estos se encuentran el funcionamiento de una gráfica, el lenguaje C o la importancia y potencial de la ejecución paralela, además de la arquitectura y lenguaje CUDA, sobre la cual tenía completo desconocimiento.

Gracias a esto he desarrollado capacidades de aprendizaje autodidacta y entendido la importancia de la documentación de software, elemento a partir del cual se puede aprender sobre cualquier tecnología desconocida.

Además de lo mencionado anteriormente, me he dado cuenta de lo crucial que es para un proyecto la planificación inicial, dividiendo toda la realización del proyecto en tareas y pautas a seguir para el correcto desarrollo del mismo.

11.3 Vías de trabajo futuro

Por último, se indican posibles ampliaciones que se podrían realizarse en un futuro utilizando el programa de este trabajo de fin de grado.

Una posible aplicación gráfica que sirva como interfaz para que un usuario pueda introducir los arrays específicos que quiera ordenar y permita la visualización de los diferentes arrays resultantes.

También podría desarrollarse una aplicación que realice histogramas sobre diversos conjuntos de datos que sean representables mediante un array de números integrales. Por ejemplo en el procesado de imágenes, un histograma de los valores píxeles de la imagen puede indicar características como la dominancia de colores o la distribución de intensidades.

12 Referencias

- [1] NVIDIA Corporation & affiliates, *CUDA Toolkit Documentation 12.2* (2023)
- [2] C. Represa Pérez, J. M. Cámara Nebreda, P. L. Sánchez Ortega, Universidad de Burgos, “Introducción a la programación en CUDA”, v 3.1. (2016)
- [3] Espeso, P - *Las GPU como pasado, presente y futuro de la computación* (2014)
- [4] Apuntes de “Hardware de programación específica”, proporcionados por el profesor Carlos Castro Miguéns en el curso 2021/2022
- [5] R. Davidson, Steven, *Blog Con Clase | Ordenación por cuenta* (2001)
- [6] Dr. Suhas Patil *Test Driven Development (TDD): ¿Qué es el desarrollo basado en pruebas (TDD)?* (2023)
- [7] C. Represa Pérez, *CUDA C/C++: Guía de instalación* (2021)
- [8] NVIDIA Corporation & affiliates, *CUDA C PROGRAMMING GUIDE* (2012)
- [9] *gnuplot.inf, Gnuplot documentation* (2022)
- [10] R. Echevarría Libano, Universidad de Sevilla, *Breve introducción a gnuplot* (2017)